Exposing Probe-Resistant VPNs via Protocol Coexistence

Enrique Sobrados, Muhammad Danish, Jack Vanlyssel, Roya Ensafi†, Afsah Anwar
University of New Mexico, {esobrados720, mdanish, jcvanly, afsah}@unm.edu
†*University of Michigan, ensafi@umich.edu*

***Abstract*—Governments and ISPs have been increasingly enforcing strict censorship regulations over the last decade. This has led users to rely on VPN solutions to bypass content restrictions. In response, censors have begun blocking VPN connections through active probing and passive traffic analysis. Now, VPN providers are deploying protocols with probe-resistant features to avoid being detected by censors.**

In this paper, we find that commercial VPN providers often deploy a collection of VPN protocols—including probe-resistant protocols like WireGuard and probe-sensitive protocols like IKE—on the same server. We show this coexistence leads to a weak-link vulnerability that can be abused by an adversary (censor) to identify and block VPN proxies. Moreover, we find that the weak-link vulnerability is unfortunately overlooked and prevalent in most popular VPN providers including ProtonVPN, NordVPN, and Surfshark. In our analysis of 13 VPN applications, we discover that the weak-link affects 90.1% of analyzed servers hosting probe-resistant protocols, with 9 providers exhibiting it in more than 95% of their servers. We find that the weak-link affects servers running WireGuard at least 90% of the time, while NordWhisper, a censorship-oriented proprietary VPN protocol, is affected in 99.2% of servers. Further, we analyze four providers deploying double-hop topologies and find that their entry servers remain exposed to the weak-link vulnerability, providing censors with a directly reachable point for identifying and blocking the connection. Finally, we demonstrate how censors can exploit the weak-link vulnerability to identify additional VPN infrastructure, including otherwise probe-resistant servers.

1. Introduction

Governments and Internet Service Providers (ISPs) have been increasingly enforcing censorship regulations over the last decade [1]–[6]. These regulations have led users to seek and adopt commercial VPN services to bypass restrictions [7]. As a result, censors are now proactively detecting and blocking VPN usage. For example, the Turkish government recently blocked 16 commercial VPN providers [8]. Similar reports have emerged from multiple countries, including India, Russia, Cuba, Venezuela, and Uganda, exhibiting the global relevance of the problem [9]–[13].

To detect VPN connections, censors often rely on active network analysis [14], [15]. Such techniques employ active probing to identify VPN servers by collecting server metadata, service banners, certificates, and other externally observable network characteristics to infer VPN usage [16], [17]. In practice, the low cost of active probing-based blocking techniques makes them easy to implement and therefore widely accessible to censors of all sizes [18], [19]. In response, VPN providers, such as Proton VPN, have widely advertised their enhanced resistance to detection and censorship circumvention capabilities (Figure 1).

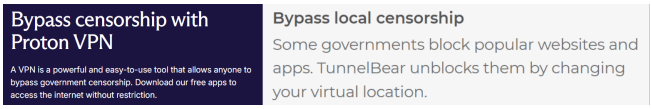


Figure 1: Circumvention advertising examples [23], [24].

To ensure these protections, VPN providers are increasingly adopting VPN protocols designed to resist active probing. For example, newer OpenVPN deployments can silently drop packets from unauthorized clients [20]. Similarly, WireGuard is designed to remain unresponsive to unauthenticated probes and does not reveal its presence to peers that cannot successfully authenticate [21]. Consistent with these design goals, prior work has found such protocols difficult to identify through active measurements in the wild, even when conducting Internet-scale scans [22].

In this paper, we explore whether the increased adoption of probe-resistant protocols by commercial VPN services prevents censors from fingerprinting their VPN infrastructure. Surprisingly, **we find that VPN providers frequently co-deploy probe-resistant protocols alongside protocols that remain susceptible to fingerprinting**. This coexistence could potentially create a weak-link vulnerability that can be abused by censors to detect and block the VPN infrastructure. To further investigate the weak-link vulnerability, we focus on three key questions: (1) *How prevalent is the weak-link vulnerability in popular VPN applications?* (2) *To what extent does the weak-link vulnerability undermine the effectiveness of probe-resistant VPN protocols and modern VPN topologies?* and (3) *How can an adversary weaponize the weak-link vulnerability to fingerprint VPN providers' infrastructures at scale?*

Our findings show that the weak-link vulnerability represents an ecosystem-level problem that enables the fingerprinting of provider-wide VPN infrastructure. We explore the implications of the weak-link vulnerability through a comprehensive assessment of 13 popular VPN providers, including NordVPN, McAfee, and Surfshark. By reverse engineering the VPN applications' APIs, we enumerate

72,887 servers and identify the protocols they support to assess the weak-link vulnerability. **We find that 45,767 (83.41%) of the 54,869 scanned VPN servers are susceptible to fingerprinting** by censors, impacting more than 500 million users. Concerningly, **9 out of 13 providers have over 95% of their infrastructure vulnerable to the weak-link**. We unveil worrying deployment practices causing this vulnerability, such as providers co-deploying WireGuard with probe-sensitive protocols over 90% of the time. In addition, we find that **censorship-oriented protocols like Shadowsocks and Xray are also impacted by the weak-link, making them ineffective for their primary use case**. Further, we find that NordVPN's circumvention-oriented proprietary protocol (NordWhisper) is vulnerable to the weak-link 99.2% of the time.

VPN providers are also adopting double-hop routing to provide additional privacy and security properties to VPN connections, including greater resistance to censored environments. We find that exposed entry nodes allow censors to identify and block double-hop connections even when exit nodes remain protected. We also contextualize these findings using providers' claims of obfuscation, protocol security, and double-hop deployments. Finally, through case studies of three additional VPN applications, we demonstrate that the weak-link vulnerability can be leveraged to further impact servers that are technically probe-resistant.

In summary, our main contributions are:

- We develop a data collection pipeline to enumerate VPN infrastructure by reverse engineering VPN applications' APIs, allowing us to identify 72,887 servers across 13 popular VPN providers (§3 and §5).
- We measure the susceptibility to fingerprinting of VPN infrastructures and analyze how their exposure contradict obfuscation claims (§4.1 and §5.1).
- We expose the weak-link vulnerability in servers hosting probe-resistant VPN protocols, enabling infrastructure-level detection despite protocol-level obfuscation (§4.2 and §5.2).
- We show that the weak-link vulnerability affects double-hop configurations by exposing entry nodes that can be directly blocked by censors (§4.3 and §5.3).
- We demonstrate that the weak-link vulnerability enables large-scale identification and blocking of provider-specific VPN infrastructure, and responsibly disclose our findings to affected providers (§6 and §7).

Call for Action: Our results show that the weak-link vulnerability is a systemic issue in the VPN ecosystem. However, unlike other security challenges, which require complex solutions, protecting against the weak-link is straightforward: Do Not Coexist. As such, providers should remove support for fingerprintable protocols. When this is not feasible due to backward compatibility, they should isolate and avoid deploying fingerprintable protocols alongside their probe-resistant counterparts. We have reached out to the investigated VPNs and we will continue reaching out to additional providers to make them aware of our findings.

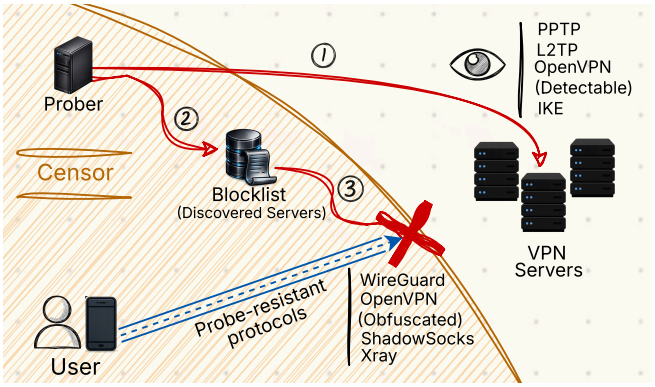


Figure 2: The Weak-Link Vulnerability: censors detect VPN servers via observable protocols and then blocklist them, affecting users connected through probe-resistant protocols. Figure inspired by [25].

2. Background and Overview

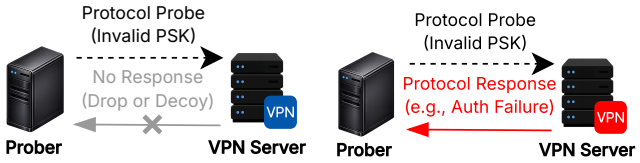
We begin by describing the censor threat model and define *strong vs weak* protocols from a server-side observability perspective, which we use throughout the paper. Then, we outline our approach to examine the weak-link vulnerability and evaluate its impact on commercial VPN infrastructures.

2.1. Censor Threat Model

We assume a realistic censor (i.e., ISP or government) that seeks to identify VPN connections within its controlled autonomous systems (AS). The ultimate goal of the attacker is to detect and disrupt VPN connections by blocking VPN servers [8], throttling Internet speeds [26], and applying differential treatment [27].

The censor can observe network traffic within its controlled networks to identify the remote servers contacted by clients and subsequently perform follow-up probes against those servers. In particular, the censor is capable of crafting stateless probes with protocol-specific signatures to identify probe-sensitive VPN protocols, following techniques described in prior work [18], [28]. Then, the censor exploits the weak-link vulnerability to impact servers hosting probe-resistant protocols, as illustrated in Figure 2: (1) the censor probes servers that expose probe-sensitive protocols; (2) once a VPN service is identified, the corresponding server IP is added to a blocklist; and (3) subsequent client connections to the flagged IP are blocked.

This paper focuses on detecting VPN infrastructures through active probing. Although censors may use other techniques (e.g., deep packet inspection) to detect VPN connections, such attacks are beyond the scope of this work. While undetectable communications are essential for a VPN to circumvent censorship, resistance to active probing is equally important, as adversaries can otherwise identify and block its infrastructure, motivating the focus of our work.



(a) Strong Protocol Auth (b) Weak Protocol Auth

Figure 3: Packet authentication failure semantics in VPN protocols under invalid pre-shared keys (PSKs). Under authentication failure, weak protocols respond with protocol-specific responses, whereas strong protocols provide non-informative responses (i.e., silent drop or decoy).

We assume the censor does not rely on sophisticated probing techniques that detect "probe-resistant" VPN protocols [17]. While some nation-state censors, such as the GFW of China, possess the resources to employ state-of-the-art probes to detect obfuscated protocols [16], our threat model focuses on low-cost, scalable, and easy-to-deploy techniques that can be adopted by censors of all sizes [18].

2.2. Strong and Weak Protocols

Throughout this paper, we use the terms *strong* and *weak* to refer to VPN protocols based on their server-side susceptibility to fingerprinting, which enables censors to classify them as VPN infrastructure. Specifically, as illustrated in Figure 3, we classify protocols according to their behavior when faced with unauthenticated probes or invalid pre-shared keys (PSKs). In particular, we distinguish between *non-informative* responses, which reveal no protocol-specific information (Fig 3a), and *informative* responses, which expose protocol-specific semantics (Fig 3b). Table 1 introduces the strong and weak protocols covered in this work, describing their probe resistance behavior under unauthenticated probes, and whether they are modern or deprecated.

We define *strong* protocols as those that are probe-resistant to Internet scanners. They exhibit non-informative behavior, such as silently dropping packets or mimicking other protocols, when receiving unauthenticated probes. This includes modern VPN protocols such as WireGuard and OpenVPN* (an enhanced OpenVPN configuration using the `tls-auth`/`tls-crypt` directives [20]), proxy-based systems such as Xray and Shadowsocks, or proprietary solutions (NordWhisper). We emphasize that *strong* does not imply cryptographic unbreakability, but rather the presence of obfuscation semantics that hinder protocol identification. As a result, strong protocols make it more difficult to detect VPN infrastructure, enabling their use for bypassing content restrictions and censorship. In Table 1, these protocols are marked with ✓ in the PR column.

In contrast, *weak* (probe-sensitive) protocols do not implement obfuscation and therefore expose informative responses under active probing. Using protocol-specific probes, these protocols can be fingerprinted due to their deterministic behavior (e.g., returning protocol-specific messages or errors). This category includes both modern but still fingerprintable protocols, such as OpenVPN⁻ (the default configuration of OpenVPN which lacks control-plane authentication) and IKEv2, and deprecated protocols, such as IKEv1, PPTP, and L2TP, which are no longer recommended due to known vulnerabilities [29]–[31]. In Table 1, these protocols are marked with × in the PR column.

Protocol	Class	PR	Probe Behavior
WireGuard	Modern	✓	Silent drop [21]
OpenVPN*	Modern	✓	Silent drop [20]
OpenVPN⁻	Modern	×	Exposed TLS handshake [32]
Xray	Modern	✓	HTTP decoy [33]
Shadowsocks	Modern	✓	Silent drop [34]
NordWhisper	Modern	✓	Undocumented/proprietary [35]
IKEv2	Modern	×	Visible handshake [36]
IKEv1	Deprecated	×	Visible handshake [37], [38]
L2TP	Deprecated	×	Control-channel reply [39], [40]
PPTP	Deprecated	×	Control-channel reply [39], [41]

VPN Proxy/Obfuscation PR = Probe-Resistant
OpenVPN* = `tls-auth/crypt` OpenVPN⁻ = no control-plane auth

TABLE 1: Comparison of protocols based on their resistance to active probing.

2.3. Approach Overview

Understanding the weak-link vulnerability requires us to contextualize the visibility of the censors into VPN infrastructures and their ability to use active probing to fingerprint them. This necessitates that we enumerate VPN servers of commercial VPN providers and identify the underlying deployed protocols. Below, we describe our approach for identifying the vulnerability and measuring its impact on strong protocols and double-hop deployments.

We begin by describing our data collection pipeline to comprehensively enumerate commercial VPN servers and identify strong and weak protocols (§3). Afterward, we introduce three analysis dimensions of the weak-link vulnerability (§4). Next, we present our dataset selection criteria and the results of our data collection and analysis (§5). Then, we demonstrate how our findings can be leveraged to move beyond individual server exposure to impact probe-resistant servers at scale (§6). Finally, we discuss the recommendations, the real impact of our findings, and how we responsibly disclosed to the affected providers (§7).

3. Data Collection Pipeline

The goal of this section is to describe the pipeline used to collect a ground-truth dataset of VPN servers and the protocols they support, providing the foundation for our analysis of the weak-link vulnerability. To achieve this, we develop a three-stage data collection pipeline depicted in Figure 4: (1) Network Collection, to capture application layer traffic containing comprehensive server lists; (2) Server Enumeration, to systematically identify and enumerate VPN servers over time; and (3) Data Augmentation, to identify strong and weak protocols, and collect obfuscation claims to contextualize the weak-link analysis.

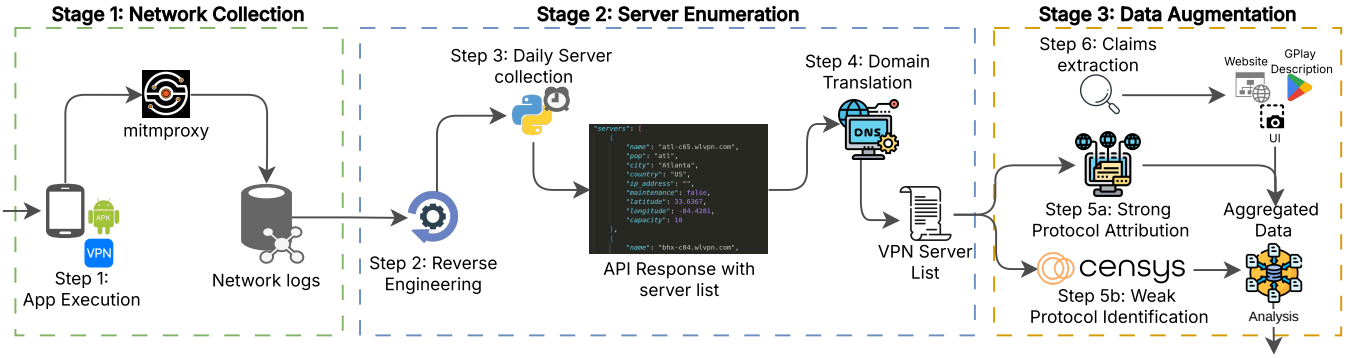


Figure 4: High-level data collection pipeline: (1) we execute VPN apps and capture traffic logs that contain server lists; (2) we reverse-engineer app behavior and automate daily server enumeration; and (3) we augment the collected servers with Internet-wide scan data to identify exposed services and attribute protocols to VPN servers.

3.1. Network Collection

To study the weak-link vulnerability, we first need a comprehensive VPN server corpus. In this stage, we extract server lists directly from VPN applications, providing a starting point for infrastructure enumeration. Prior work has enumerated VPN servers by mimicking censor behavior through active probing [18], [22]. However, this approach has several limitations as it fails to identify VPN servers that only host strong protocols or servers that apply enhanced obfuscation mechanisms. Therefore, instead of probing servers directly, we analyze VPN clients' interactions with their backend infrastructure to determine how server information is retrieved, thereby constructing a ground-truth server list.

We describe an abstract flow of how a VPN client interacts with its backend to obtain the server IP address and establish a VPN tunnel (Figure 5). First, the client contacts the backend API to request information about available servers. Second, the backend replies with server metadata, such as IP addresses, domains, and supported protocols, often including a full server list used to populate the app's interface. Third, the client connects to a selected VPN server and tunnels all user traffic. Fourth, the VPN server decapsulates the traffic and sends it to its final destination. We collect comprehensive and ground-truth lists of VPN servers from the backend API responses sent to the client.

Experimental Setup. To conduct a representative study, we utilize a device environment that captures a large portion of the VPN ecosystem. Recent reports estimate that 89% of users access VPNs from mobile devices [42], making mobile VPNs the most relevant platform for our study. Within the mobile ecosystem, in 2026, Android accounts for approximately 68% of the global mobile market [43]. Additionally, other mobile operating systems have smaller market shares (Linux) or are inaccessible for device instrumentation required for this study (iOS). Therefore, we select Android as our primary platform to analyze VPN servers.

We use Genymotion [44] to emulate a rooted Pixel XL running Android 14. Although some applications may detect emulated environments and exhibit different behavior [45], our experiments across the full app corpus show that the targeted API requests are device-independent, resulting in consistent backend responses across device architectures. In addition, we configure `mitmproxy` [46] in transparent mode to capture HTTPS traffic and use `Frida` [47] to bypass SSL pinning techniques. We also use `Frida` to bypass root detection, environment emulation checks, and debugger checks to ensure consistency with production behavior.

Step 1: App Execution. During application runtime, we identify specific events that trigger server list requests. We discovered three primary cases: (1) On startup: the client automatically sends a `GET/POST` request to the backend to populate the user interface (UI); (2) Server list view: a lazy-loading approach where the request is made when the user opens a UI view that contains all supported servers; (3) Tunnel establishment: the application is forced to request server details when initiating a VPN connection.

To ensure that the server list is sent to the client, we attempt to trigger these requests across all applications. For premium ones, we attempt to trigger the request with the first two events when possible. If the server list is transmitted, we then purchase the paid plan for subsequent analysis.

3.2. Server Enumeration

We utilize VPN APIs to systematically enumerate each provider's infrastructure. However, relying on a single server list response may limit our visibility into VPN infrastructures: providers may only transmit a subset of their total server fleet or provide a fixed set of results (e.g., only four servers per response). To mitigate these constraints and ensure maximum coverage, we perform these requests periodically (i.e., daily). This longitudinal approach allows us to aggregate fragmented responses into a comprehensive map of their VPN infrastructure. Because each VPN provider employs a distinct backend architecture, the request logic used to retrieve server information must be reverse-engineered. We therefore begin by describing our reverse-engineering process, followed by our methodology for daily server collection.

Step 2: API reverse engineering. We perform a manual examination of the network traffic to identify how the mobile

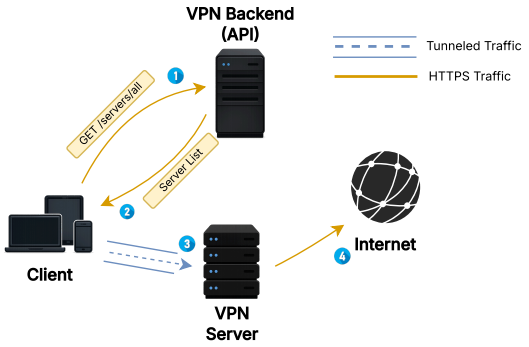


Figure 5: High-level overview of how a VPN works.

```
{
  "servers": [
    {
      "hostname": us-abq-01.example.com,
      "ip": 192.0.2.10,
      "country": US,
      "city": Albuquerque,
      "protocols": [
        WireGuard,
        OpenVPN,
        IKEv2
      ],
      "load": 42
    }
  ]
}
```

Figure 6: Illustrative VPN backend API response containing server metadata and supported protocols.

client requests server data. This step is necessary because there is no generic method for infrastructure extraction across different providers. For example, some applications utilize unauthenticated `GET` requests to retrieve server lists, while others require bearer tokens or API keys. By decomposing the communication logic, we ensure that we can accurately replicate the client's behavior to fetch comprehensive server data.

We recognize VPN servers by identifying the specific request to the API endpoint that returns server metadata (e.g., `api.examplevpn.com/servers/all`). In the response body, we look for groups of IP addresses or domain names with descriptive characteristics, such as server location or available protocols (see Figure 6). Once the presence of infrastructure data is confirmed, we look for all the required parameters, such as Bearer tokens or API keys, required to make the request.

Step 3: Daily Server Collection. To ensure accurate replication of the original requests, we use the Python `requests` library and define identical headers, URL parameters, and body payloads for the server list request. When these parameters are obtained dynamically, we replicate all the previous requests needed for their generation. For example, if an endpoint requires a dynamically generated API key for authorization, our script first calls the appropriate endpoint to retrieve and assign that value. This guarantees that our scripts do not use expired tokens or have insufficient parameters that lead to inaccurate responses. For applications that send region-specific server lists, we iterate the country parameter to capture a comprehensive list of available servers across all regions. To ensure our requests do not create rate-limiting issues or affect the API endpoints, we follow ethical recommendations by implementing five-second delays between requests [48], [49].

Step 4: Domain Translation. While the majority of VPN applications provide IP addresses directly in their API responses, some utilize domain names as server identifiers. This approach allows dynamically changing IP addresses or implementing load balancing while maintaining a consistent identifier. In these cases, we resolve each domain to its IP address. To account for domains that resolve to multiple addresses or utilize IP rotation, we perform DNS resolutions for each domain every 10 seconds. We terminate the collection for a specific domain after 30 seconds without discovering any new IP.

3.3. Data Augmentation

Quantifying the weak-link vulnerability requires understanding the strong and weak protocols running on the VPN servers. In this subsection, we explain how we identify these protocols and collect providers' obfuscation claims to contextualize our findings.

Step 5a: Strong Protocol Attribution. The attribution of strong protocols to VPN servers is a key aspect of this work. Since strong protocols implement probe-resistant mechanisms, it remains challenging to attribute their availability to a specific infrastructure. Consequently, there is limited understanding of the real-world adoption of these protocols. To map strong protocols to VPN infrastructure, we first inspect the network logs for patterns indicating the protocol specificity of each server (see Figure 6). For example, when an API endpoint exposes server metadata—such as location, user load, or domain names—it also includes the supported protocols, which we use to construct a map of potential protocol usage. We then confirm protocol usage through different approaches: identifying application-layer packets that link a strong VPN protocol to a specific server or detecting protocol-related public keys to infer the protocol in use. For example, during a WireGuard connection, the network response included a `Curve25519` key that was later observed being used by a WireGuard process. Additionally, we analyze cached protocol configuration files containing server addresses, as well as system logs (i.e., logcat), to extract protocol-specific information that corroborates the use of these protocols.

Step 5b: Weak Protocol Identification. Similar to previous studies [50]–[54], we use Censys [55], an Internet-wide scanner, to obtain information about open services in the collected VPN servers. Censys implements specialized probes to detect weak VPN protocols, making it relevant to the scope of our work. We query the Censys dataset for our identified VPN servers to extract open ports, service names, and Censys tags used for infrastructure classification like

"VPN" and "Proxy". Crucially, we take advantage of their historical dataset to ensure scan results correspond to the period when the servers were actively managed by the VPN providers and avoid mischaracterizing their infrastructure. With this approach, we approximate censors' active probing capabilities for detecting these protocols. It is important to note that Censys does not detect the strong VPN protocols described in Table 1.

Step 6: Claims Extraction. Commercial VPNs prominently advertise censorship circumvention assurances to attract users [56]. Yet, the truth behind these claims is often opaque as they depend on infrastructure-level choices—including whether servers can be identified and targeted by adversaries. To contextualize our analysis, we manually extract claims that characterize the provider's assurances from three different sources: (1) screenshots of the applications' UI, (2) the provider's official website and support documentation, and (3) Playstore descriptions.

4. Analysis

After enumerating VPN servers and their available protocols, we examine the implications of the weak-link vulnerability from three key dimensions: (1) the fingerprintability of servers operated by popular VPN providers (§4.1); (2) the impact of the weak-link vulnerability on infrastructure hosting strong protocols (§4.2); and (3) the extent to which the weak-link vulnerability affects the observability of double-hop deployments (§4.3).

4.1. Infrastructure Observability

As VPNs have evolved to circumvent strict censorship measures, they are increasingly expected to resist detection by censors. This protection relies heavily on preventing fingerprinting of the VPN services hosted on their servers. Failing to provide effective obfuscation can expose users to differential treatment attacks [27], bandwidth throttling [26], and legal repercussions [57]. These issues are particularly concerning, as users place their trust in VPN providers to avoid these risks.

In this analysis, we examine the fingerprintability of servers operated by popular VPN providers. Specifically, we check whether providers maintain deployments that resist active probing. To this end, we utilize an off-the-shelf active probing service (Censys) to understand the visibility of VPN servers to a censor. We emphasize the identification of three critical pieces of information: (1) the use of standard ports by VPN protocols, such as 500/4500 (IKE), 1194 (OpenVPN), 1701 (L2TP), and 1723 (PPTP); (2) the adoption of protocols on non-standard ports; and (3) the classification of VPN infrastructure by Internet-wide scanners using a "VPN" label. Finally, we contextualize our results using providers' claims of *server obfuscation* to assess whether they uphold their guarantees of undetectability.

4.2. Strong-Weak Coexistence

In the previous subsection, we quantify infrastructure observability from a censor perspective. In this analysis, we look deeper into deployment practices to understand whether providers allow strong protocols to coexist alongside weak protocols, creating the weak-link vulnerability.

VPN providers are increasingly adopting strong VPN protocols, such as WireGuard and OpenVPN*, to take advantage of their improved security and obfuscation properties [58]. In parallel, proxy protocols such as Shadowsocks and Xray are also gaining adoption due to their built-in obfuscation capabilities, refined through long-term use in censorship-circumvention settings [15], [59]. This has led VPN providers to discontinue support for deprecated protocols, such as PPTP, L2TP, and IKEv1 [31], [60]. Additionally, modern yet fingerprintable protocols, such as IKEv2 or OpenVPN⁻, are being phased out [61], [62].

Unfortunately, to this day, despite claims of discontinuing weak protocols, we find that VPNs continue to support these weak protocols, often deploying them alongside strong protocols. This coexistence can undermine the probe-resistant properties offered by strong protocols. As explained in §2.1, a user may establish a VPN connection using WireGuard (strong) while the same server also supports IKEv2 (weak). Although the censor cannot directly detect WireGuard, the coexistence of IKEv2 may expose the server to detection and ultimately lead to its blocking.

To study the prevalence of the weak-link vulnerability, we pair each strong protocol with each weak protocol observed on the same VPN server, generating `(strong, weak)` tuples. For example, if a VPN server hosts WireGuard (s) and IKEv2 (w), we generate the tuple (s, w). We then quantify the coexistence ratios for each strong-weak protocol pair by calculating the proportion of servers with strong protocols that simultaneously deploy weak protocols. For each strong protocol s and weak protocol w, we compute the coexistence ratio as:

$$R_{s,w} = \frac{|S_s \cap S_w|}{|S_s|},$$

where S_s is the set of servers hosting strong protocol s, and S_w is the set of servers exposing weak protocol w. Using these ratios, we examine which strong protocols are mostly affected. Finally, we contextualize our findings using VPN providers' claims about *protocol security*.

4.3. Double-Hop Exposure

Double-hop architectures are designed to provide additional security and privacy benefits, and some VPN providers advertise them as offering enhanced resilience in censorship-related scenarios [63]. As shown in Figure 7, a double-hop setup routes traffic through two distinct servers: the client first connects to an entry server (IP A), which redirects traffic to an exit server (IP B). A web service then sees the incoming request as originating from IP B.

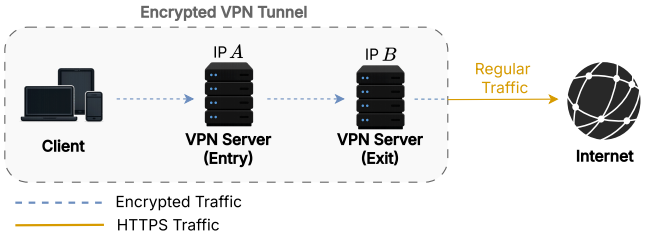


Figure 7: Double Hop Architecture: The VPN client (User Device) connects to an Entry VPN Server with an IP address A. Then the VPN redirects traffic to a second VPN server with IP B, which will be the public IP of the connection.

The double-hop design separates the client-facing entry server from the public-facing exit server, ensuring that no single endpoint directly observes both sides of the connection. VPN providers are starting to implement double-hop as their default network routing. From a censorship perspective, however, the entry server remains particularly important because it is the endpoint directly contacted by the user. If the entry node exposes a fingerprintable protocol, a censor observing the user's network can identify and block that server even when the exit node remains protected. We therefore investigate how weak-link exposure differs between the entry and exit components of double-hop deployments.

Topology Inference. For our analysis, we define a double-hop architecture as one where $A \neq B$. Because the client only needs the entry IP (A) to establish a tunnel, the exit IP (B) is rarely included in API responses. Consequently, using only strong protocols, we perform at least 50 manual connections to map A to B. To identify entry IP A, we examine the network packet generated during the server connection phase. Specifically, when a user selects a specific server or clicks the "connect" button, the application typically requests mandatory connection data from the API, such as keys, credentials, and the endpoint IP (A). To find exit IP B, during a tunnel connection, we query popular IP lookup services (i.e., `whatsmyipaddress.com`, `api.ipify.org`) and check the application's built-in UI lookup (some applications show the public IP B in their UI when a VPN connection is established). We exclude any app where $A = B$ from this specific analysis.

After forming these (A, B) pairs, we query Censys to identify open services, ports, and labels for both endpoints. Finally, we examine whether entry and exit nodes expose weak VPN services to determine how the weak-link vulnerability affects the observability of different components of double-hop deployments.

5. Results

This section presents the results of our systematic analysis of the weak-link vulnerability in commercial VPN infrastructure. We begin by describing the VPN application dataset used in our study. Next, we analyze these applications to assess the observability of their servers (§5.1), evaluate the prevalence of the weak-link vulnerability within their infrastructures that undermines their censorship circumvention claims (§5.2), and examine their impact on emerging VPN topologies (§5.3).

Filtering Step	#Apps	Filtering Step	#Apps
Initial App Corpus	51	Encrypted Payloads	13
Less 1M	2	Lack Protocol Info	3
Strong Anti-Analysis	19	Server List Reliability	1
Final App Corpus		**13**	

TABLE 2: VPN Application Selection and Filtering. We list the number of apps that meet the filtering steps.

VPN App Selection. We identified an initial pool of 51 VPN applications by manually querying for "VPN" in Google Play Store. We retain only popular VPN applications (i.e., apps with more than one million installs) to focus on providers that are representative of the broader VPN ecosystem and whose deployment practices affect a large user population. We then execute each app in the network collection module.

We remove any app that crashes during startup or employs strong anti-analysis techniques that cannot be bypassed by standard `Frida` tools. Consequently, we reverse-engineer the network traffic of 30 apps in our dataset to identify API requests with the server list. Interestingly, some applications add an extra encryption layer to their response payloads, even though the responses are already transmitted over TLS. While we can infer the responses containing the servers, we cannot obtain the list of servers in plaintext. We therefore exclude apps that encrypt their responses.

After these exclusions, we obtain a list of 17 VPN apps. For these apps, we identify the endpoints that return the server list to the client. Among these, three do not expose the information required to attribute both strong and weak protocols to their servers, which is necessary for our analysis. We therefore remove them from our dataset.

Server List Reliability. For every app, we perform a confirmation step to ensure the reliability of the server list. Specifically, we manually establish 10 VPN connections and inspect network traffic to ensure that connected servers are included in the server list. During this validation step, we find that one app publishes a server list but never uses those servers in practice. Therefore, we remove that application from the dataset.

We summarize the filtering process in Table 2. Table 3 further shows the final dataset containing 13 applications with more than 500 million cumulative installations.

Daily Server Collection. We run the dataset collection pipeline from February 2025 to November 2025 (10 months). In total, we collected 72,887 VPN servers across 13 providers (see Table 3). We observe that McAfee Security contributes the largest number of servers (40,850), representing ≈56% of our dataset. NordVPN has the second-largest infrastructure (8,922 servers) and one of the largest user bases (100M+ installs). In contrast, Instabridge—despite being the most downloaded app in our dataset—maintains a comparatively small server list (362). This may be be-

Application	Servers	Installs	Internet Scanner	
			Scanned	Detected (%)
McAfee Security	40,850 (56.05)	50M+	22,858	20,341 (88.99)
NordVPN	8,922 (12.24)	100M+	8,917	7,496 (84.06)
CyberGhost VPN	5,436 (7.46)	10M+	5,436	4,312 (79.32)
ProtonVPN	3,756 (5.15)	50M+	3,756	1,175 (31.28)
Surfshark	3,472 (4.76)	10M+	3,472	3,445 (99.22)
IPVanish	3,275 (4.49)	10M+	3,275	3,275 (100.00)
Bitdefender VPN	3,275 (4.49)	1M+	3,275	3,275 (100.00)
VPN99	1,760 (2.41)	1M+	1,760	345 (19.60)
PureVPN	1,118 (1.53)	5M+	1,097	1,093 (99.64)
Instabridge	362 (0.50)	100M+	362	361 (99.72)
Browsec	332 (0.46)	10M+	332	329 (99.10)
Zoog VPN	172 (0.24)	1M+	172	169 (98.26)
VPN Germany	157 (0.22)	1M+	157	151 (96.18)
Total	**72,887 (100.00)**	**500M+**	**54,869**	**45,767 (83.41)**

TABLE 3: Distribution of VPN servers across applications, including the number of installations and detection coverage. *Scanned* denotes the number of servers queried using Censys. *Detected* denotes the number of servers that deploy a weak protocol and are therefore fingerprintable. Highlighted in red are instances with more than 95% detection rate.

cause VPN functionality is not the app's primary feature. The mean and median server counts are 5,607 and 3,275 servers, respectively, highlighting the large-scale and real-world representation of the VPN ecosystem.

Strong Protocol Attribution. Recall that we perform a protocol confirmation step, where we corroborate protocol usage at the system and UI level (Step 5a in §3.3). In cases where this corroboration was uncertain, we conservatively refrain from attributing the protocol to the server. Additionally, when a server is initially identified as OpenVPN (through client-side analysis) but is later fingerprintable by Censys, we classify it as OpenVPN^{-}; otherwise, we consider it as OpenVPN*. As a result, we successfully attributed at least one strong protocol to 26,212 servers across all applications. We find WireGuard being used in 11 out of 13, OpenVPN* in 6, ShadowSocks in 4, and Xray in 3 applications. The attribution results are shown in Table 5 in the "Servers by Protocol" and "Strong Protocol" columns.

Notice that we only attribute strong protocols to 26,212 out of 72,887 servers. While this decrease seems considerable, this is mainly caused by apps that use domain names as identifiers, where protocol details in the network logs are provided per domain. In our conservative approach, we limit attributing every resolved IP from the domain to the domain-specific protocol. Instead, we only attribute a strong protocol when the protocol is explicitly listed together with the IP. We argue that this is critical for the accuracy of our findings.

Weak Protocol Identification. We query each of the identified VPN servers on Censys and retrieve the snapshot for the day the IP address first appears in the server list. Due to Censys' credit limits and the number of servers operated by McAfee, we could not cover every individual server across all applications. Since we prioritize understanding the weak-link over all the apps, we chose to randomly sample 56% of McAfee servers while keeping the coverage for the other applications unchanged. Even after sampling, McAfee remains the largest server list for any single application. Additionally, we exclude servers for which Censys scan retrieval failed (5 from NordVPN and 21 from PureVPN), as the absence of scan data prevents us from reliably determining whether these servers are probe-resistant. In total, we obtain exposed services for 54,869 (≈75%) VPN endpoints.

Category	Claims (Examples)	App
Detection & Blocking	*"Hide the fact that you're using a VPN"*	Surfshark
	"Bypass internet censorship"	Browsec
Protocol Security	*"fastest and most trusted VPN protocols"*	Zoog VPN
	"Stay secure and flexible with cutting-edge VPN protocols"	PureVPN
Double Hop	*"multi-hop VPN for strict internet regulation"*	NordVPN
	"A double VPN lets you enjoy the internet freely"	Surfshark

TABLE 4: VPN security claims collected from app GUIs, Google Play Store descriptions, and official websites.

Claim Analysis. Understanding the claims made by providers is important because marketing exposure shapes VPN users' mental security models [64], [65]. We find numerous claims regarding infrastructure observability, secure protocol adoption, and guarantees of double-hop topologies. Table 4 lists two representative examples for each analysis category. Concerningly, Instabridge provides minimal information about its VPN functionality. Since VPNs create a transfer of trust, this lack of transparency is alarming.

Next, we show the results of our three analysis verticals and compare our findings with the security guarantees offered by VPN providers.

5.1. Infrastructure Observability

In this subsection, we evaluate the fingerprintability of servers operated by VPN applications. Particularly, we showcase how the presence of weak protocols enables fingerprinting, thereby violating security guarantees.

We present the detection rates at which VPN servers can be classified as VPN infrastructure in Table 3. Overall, 83.41% of the scanned VPN servers expose at least one weak VPN protocol, indicating that most of their VPN infrastructure remains readily enumerable by censors. Alarmingly, eight applications (highlighted in red in Table 3) exceed a 95% detection rate, highlighting the pervasive absence of effective obfuscation. While a large server footprint may suggest more investment in user-facing infrastructure, we find that McAfee remains detectable in 89% of cases despite operating over 40,000 servers and serving more than 50 million users. Similarly, NordVPN exposes 84% of its servers despite its widespread popularity among VPN users.

Infrastructure Sharing and Exposure. Additionally, we find that VPN99 exhibits the lowest detection rate (≈20%). We later examined VPN99's servers and discovered that 97% of their detectable servers (345) are shared with Instabridge, which has a 99% detection rate. This suggests that infrastructure sharing can lead to unintended

exposure: a provider that deploys weak VPN services on shared endpoints makes other apps' infrastructure discoverable too, even if they employ only strong protocols. This is particularly concerning since prior work has found that many VPN applications often share the same infrastructure [66], which can create collateral damage to other apps.

Use of Default Ports. Even worse, we find that all IKEv2, IKEv1, L2TP, and PPTP services are deployed on their default ports, with no obfuscation mechanisms in place. OpenVPN$^-$ is the most prevalent protocol in the dataset, with 121,996 active services, and in several cases, it appears on more than three distinct ports on the same server. IKEv2 follows with 42,652 active services, while IKEv1, L2TP, and PPTP account for 1625, 276, and 167 services, respectively. Alarmingly, this behavior has been widely studied, where it remains critical to not deploy VPN protocols on standard ports [18], [67].

Exposure to Censors. The prevalence of weak protocols such as IKEv2 and OpenVPN$^-$, across major VPN providers, enables censors to fingerprint their servers. A major contributing factor to this visibility is the widespread use of default ports, which suggests a lack of obfuscation intentions. Even worse, many servers are already classified as VPN infrastructure by Censys, creating a direct vector for censors to leverage this information for VPN blocklisting.

Claims Analysis. Our findings directly contradict common *Obfuscation* claims by VPN providers, as listed in Table 4. For instance, Surfshark states "*hide the fact that you're using a VPN*", but a censor can fingerprint 99.22% of their servers. Similarly, Browsec's assurance of "*bypass internet censorship*" fails when they deploy weak protocols on 99.1% of their servers. This issue may also expose users to real-world consequences in jurisdictions where VPN usage is restricted. For example, in 2023, a Chinese programmer was reportedly fined 1 million yuan for using a VPN [57]. Overall, advertising obfuscation without prioritizing obfuscation is misleading and potentially harmful to users.

Takeaway: Despite widespread claims of censorship resistance, approximately 83% of VPN servers in our dataset are vulnerable to detection by censors, with 8 applications exceeding a 95% detection rate. This shows that VPN providers are vulnerable to large-scale fingerprinting and targeted blocking.

5.2. Strong-Weak Coexistence

In the previous subsection, we analyzed the infrastructure observability from a censor's perspective. However, the adoption of strong protocols should make servers resistant to active probing, reducing their overall exposure to censors. In this subsection, we aim to understand why VPN servers remain detectable despite the adoption of strong protocols.

We find that this exposure stems from the weak-link vulnerability, which arises from the coexistence of strong and weak protocols on the same servers. Table 5 shows, for each VPN application, the strong–weak coexistence ratio and the corresponding *dominant Weak-Link* associated with the strong protocols. The *Unique Weak-Link* column shows the number of unique servers affected by the weak-link vulnerability. Additionally, we highlight (in red) the applications whose dominant weak-link is a deprecated protocol.

Across all applications, the overall prevalence of the weak-link vulnerability is 90.1% of all the attributed servers. Alarmingly, 9 providers exceed a vulnerability rate of 95%, indicating that it is uncommon among providers to deploy only strong protocols. On the other hand, VPN99 exhibits the best deployment behavior by having a relatively small vulnerability rate (27.4%).

Dominant Weak-link. For six applications, the dominant weak-link is IKEv2. Although this protocol is commonly marketed as "secure" due to its strong cipher negotiation and efficient tunneling [68], [69], its initial handshake behavior enables straightforward infrastructure identification, making IKEv2 deployments visible to simple probes [36]. However, we observe that some VPN apps are starting to move away from this protocol. For example, NordVPN has removed IKEv2 from its protocol offering in its UI while still listing it as supported on its website, suggesting an ongoing deprecation from the application.

The second most frequent weak-link is OpenVPN$^-$, appearing alongside strong protocols on more than 10K servers. In practice, providers rarely provide descriptions that distinguish the use of OpenVPN* (i.e., strong configuration with `tls-auth`/`tls-crypt`) from OpenVPN$^-$ (lacking control-plane HMAC authentication). This has operational impacts, as OpenVPN$^-$ can be fingerprinted and can enable DDoS attacks, whereas OpenVPN* mitigates unauthenticated probing by silently dropping packets that fail PSK-based authentication [20]. Their distinction is relevant to users: if a user knows that a server uses OpenVPN$^-$ instead of OpenVPN*, they can prioritize selecting protocols with real probe-resistant properties (e.g., WireGuard).

Impact on Modern Protocols. We present the accumulated strong and weak coexistence counts across all applications in Figure 8. We find that WireGuard is coexisting with weaker alternatives in 22,350 out of 23,928 servers (≈93%). Even worse, across four applications, WireGuard servers are vulnerable 100% of the time. Concerningly, this behavior can harm users who choose this protocol, especially since many providers market it as the "best" VPN protocol [70], [71]. Hence, its high coexistence with protocols such as IKEv2 (21,902 instances) and OpenVPN$^-$ (10,022 instances) completely undermines one of its core design principles, which is to remain silent to unauthenticated scanners [21].

Similarly, for 3 out of 6 apps implementing OpenVPN*, its server-wide undetectability remains largely ineffective. Alarmingly, OpenVPN* servers for NordVPN, PureVPN, and Zoog VPN are vulnerable to the weak-link in 84.2%, 98.7%, and 97.1%, respectively. Thus, deploying weak protocols alongside OpenVPN* undermines one of the configuration's intended improvements: resistance to active probing. We argue that providers' efforts to implement this enhanced configuration are insufficient unless accompanied by adequate infrastructure-wide obfuscation practices.

Application	Total Servers	Strong Protocol	Servers by Protocol	Weak Protocol Identification [%]					Dominant Weak-Link	Unique Weak-Link (%)
				IKEv2	OpenVPN⁻	IKEv1	L2TP	PPTP		
NordVPN	8,833	OpenVPN*	8,833	**84.2**	-	0.6	-	-	IKEv2	7,496 (84.9)
		WireGuard	8,727	**85.3**	-	0.6	-	-	IKEv2	
		NordWhisper	7,300	**99.2**	-	0.7	-	-	IKEv2	
Surfshark	3,465	WireGuard	3,465	**98.0**	-	2.4	1.0	-	IKEv2	3,445 (99.4)
IPVanish	3,275	WireGuard	3,275	98.5	**100.0**	1.5	-	-	OpenVPN⁻	3,275 (100.0)
Bitdefender VPN	2,995	WireGuard	2,995	98.7	**100.0**	1.3	-	-	OpenVPN⁻	2,995 (100.0)
McAfee Security	2,376	WireGuard	2,376	90.7	**94.1**	1.2	-	-	OpenVPN⁻	2,280 (96.0)
VPN99	1,232	WireGuard	326	97.9	**100.0**	2.1	-	-	OpenVPN⁻	338 (27.4)
		ShadowSocks	596	54.4	**56.0**	1.3	0.5	-	OpenVPN⁻	
		Xray	433	**0.5**	0.2	-	-	-	IKEv2	
		OpenVPN*	203	-	-	-	-	-	-	
CyberGhost VPN	1,283	WireGuard	1,283	85.3	**85.6**	0.2	-	-	OpenVPN⁻	1,098 (85.6)
		OpenVPN*	185	-	-	-	-	-	-	
ProtonVPN	1,215	WireGuard	1,215	**94.2**	-	2.5	-	-	IKEv2	1,175 (96.7)
PureVPN	897	OpenVPN*	897	98.1	-	**98.7**	-	-	IKEv1	893 (99.6)
Instabridge	336	WireGuard	332	97.9	**100.0**	2.1	-	-	OpenVPN⁻	336 (100.0)
		ShadowSocks	336	97.9	**100.0**	2.4	0.9	-	OpenVPN⁻	
Zoog VPN	172	WireGuard	170	97.1	-	0.6	92.4	**97.1**	PPTP	169 (98.3)
		OpenVPN*	172	**97.1**	-	0.6	92.4	96.5	IKEv2	
		ShadowSocks	169	**97.6**	-	0.6	92.9	97.0	IKEv2	
		Xray	165	**98.2**	-	0.6	94.5	97.6	IKEv2	
VPN Germany	114	WireGuard	90	-	**96.7**	-	-	-	OpenVPN⁻	109 (95.6)
		ShadowSocks	56	-	**92.9**	-	-	-	OpenVPN⁻	
		OpenVPN*	4	-	-	-	-	-	-	
Browsec	19	Xray	19	**94.7**	-	-	-	-	IKEv2	18 (94.7)
Total	**26,212**									**23,627 (90.1)**

TABLE 5: Strong-Weak coexistence ratios. Percentages indicate the fraction of servers running a given strong protocol that are simultaneously exposed through the corresponding weak protocol. The *Dominant Weak-Link* column highlights the protocol most responsible for infrastructure detectability. *Unique Weak-Link* indicates the unique number of servers affected by the weak-link vulnerability. Highlighted in red are rows with a deprecated protocol as the dominant weak-link.

Impact on Obfuscated Protocols. We also find that censorship-oriented proxy protocols, such as ShadowSocks and Xray, are affected by the weak-link. For instance, ShadowSocks is advertised as a "tunnel proxy that helps you bypass firewalls" [72] and implements silent packet dropping when facing unauthenticated probes [15]. We find that 3 out of 4 applications have ShadowSocks running alongside weak protocols more than 90% of the time. In the remaining app (VPN99), ShadowSocks servers coexist with OpenVPN⁻ in 56% of their servers. In the same way, Xray, documented as "capable of censorship-circumvention" [73], has an average weak-link of 90% for 2 out of 3 apps. As a result, these protocols are prone to blocking caused by the weak-link vulnerability.

Even worse, NordVPN's proprietary protocol (NordWhisper) is co-deployed with IKEv2 on 99.2% of servers. This is particularly concerning as this protocol is marketed to bypass restrictive networks [35]. This highlights that even top providers like NordVPN overlook infrastructure-level protocol coexistence, leaving circumvention-oriented protocols vulnerable to the weak-link. The adoption of obfuscated and proprietary protocols is expected to increase in the future [74]. Therefore, we caution providers to eliminate any weak protocols when deploying these obfuscated protocols.

Claims Analysis. These results directly contradict *Protocol Security* claims of the VPN providers. For example, ZoogVPN states that it offers the "*fastest and most trusted VPN protocols*". Yet, we find that it still supports deprecated protocols, such as PPTP and L2TP, on 98.3% of servers hosting strong protocols. Similarly, PureVPN's assurance that users can "*stay secure and flexible with cutting-edge VPN protocols*" is undermined by its deployment of a deprecated protocol (IKEv1) alongside OpenVPN* (98.7%). These findings underscore the need for greater transparency regarding VPN providers' deployment practices and the protocols used across their infrastructure.

Takeaway: Modern protocols are increasingly adopted, but are commonly co-hosted with weak, fingerprintable, or deprecated protocols. In ≈93% of cases, WireGuard is accompanied by an insecure or discontinued protocol, making otherwise probe-resistant deployments observable. We encourage VPN providers to isolate strong protocols in dedicated VPN infrastructure.

5.3. Double-Hop Exposure

We have identified that the weak-link vulnerability is prevalent across VPN servers. In this subsection, we examine its impact on double-hop deployments, which provide multiple privacy and security benefits, including resistance to censorship environments. We identify that four VPN providers implement the double-hop routing in their normal configurations, as shown in Table 6. For Proton VPN, we

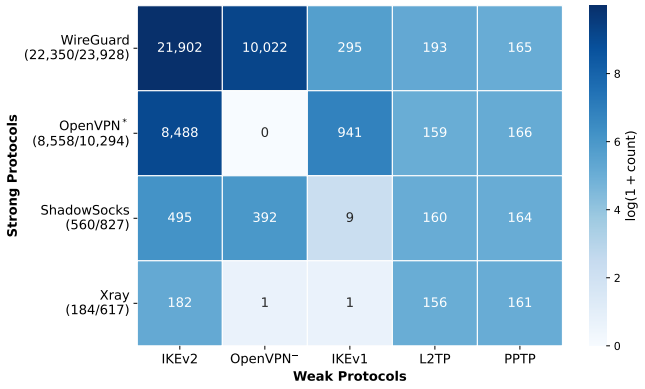


Figure 8: Distribution of strong-weak protocol coexistence. The number below each strong protocol is the number of servers vulnerable to the weak-link. Counts are deduplicated across applications; hence, WireGuard and ShadowSocks totals are lower than the per-application sums in Table 5, as VPN99 and Instabridge share servers.

find the entry and exit IPs disclosed in their API endpoint; therefore, we include all the entry-exit pairs for this app.

Entry Exposure. The detectability rates for entry and exit nodes are presented in Table 6. For 3 applications, we observe that entry servers are detectable at substantially higher rates than exit servers. For example, ProtonVPN and BitdefenderVPN maintain entry-server detection rates of 95.63% and 100%, respectively, while the corresponding detection rates for their exit servers are only 3.4% and 0%. This suggests that these deployments shield exit nodes from active probing more effectively than entry nodes. Entry-node exposure is particularly consequential because clients must directly connect to the entry server to establish the VPN tunnel. Therefore, an ISP-level adversary that identifies the entry server through a weak protocol can block the connection before traffic reaches the second hop, even when the corresponding exit node remains probe-resistant. This shows that protecting the exit node alone does not prevent blocking when the entry infrastructure remains fingerprintable.

Exit Node Exposure. Surfshark VPN differs from the other providers in that its exit nodes are also highly detectable, with an exposure rate of approximately 88%. This result shows that the weak-link vulnerability can expose both components of a double-hop deployment. While exposure of the exit node alone does not necessarily affect a user connecting through the entry node, it becomes relevant when an adversary can observe or interfere with the entry $\rightarrow$ exit or exit $\rightarrow$ final_destination paths. In such cases, exposure of the exit infrastructure provides an additional point at which the connection can be identified or disrupted.

We additionally observe strong spatial correlation in the double-hop deployments: most entry-exit combinations fall within the same /24 subnet, and in several cases the offset is a single IP. Such proximity allows an adversary that identifies an exposed entry server to narrow the search space for the corresponding exit infrastructure. By probing

Application	Entry		Exit	
	Servers	**Detected** (%)	**Servers**	**Detected** (%)
Proton VPN	756	723 (95.63)	2,234	76 (3.40)
NordVPN	50	39 (78.00)	50	0 (0.00)
Surfshark VPN	57	50 (87.72)	59	52 (88.14)
Bitdefender VPN	50	50 (100.00)	50	0 (0.00)

TABLE 6: Entry and exit server detectability rates.

nearby addresses, the adversary can enumerate candidate exit servers and expand the blocklist beyond the directly observed entry nodes. Thus, entry-node exposure can indirectly increase the visibility of exit infrastructure when both are deployed within tightly clustered address ranges.

Adversary Visibility. The impact of node exposure depends on which part of the connection an adversary can observe. Protecting the exit server can be effective against destination-side observers that only see the public exit IP, such as websites attempting to identify VPN usage. In contrast, ISP- or nation-level censors can observe the entry server contacted by the user. Consequently, even when the exit node is difficult to fingerprint, an exposed entry node can still enable the connection to be identified and blocked before traffic reaches the second hop.

Claims Analysis. Some providers advertise double-hop configurations in the context of restrictive Internet environments. For instance, NordVPN describes its offering as a "*multi-hop VPN for strict internet regulation*," while Surfshark states that "*A double VPN lets you enjoy the internet freely*." These claims motivate evaluating how double-hop deployments behave when different parts of their infrastructure remain fingerprintable. We find that 78% of the sampled NordVPN entry servers and 87.72% of Surfshark entry servers are detectable. Because users directly connect to these entry nodes, their exposure provides censors with a practical blocking point even when the corresponding exit infrastructure is less visible.

Takeaway: Four popular VPN applications deploy double-hop routing by default, yet in all cases, entry servers are still vulnerable. Because clients must directly connect to these servers, entry-node exposure provides censors with a practical point for identifying and blocking the connection. For one provider, the weak-link vulnerability can also reveal the exit servers when adversaries have visibility beyond the client-to-entry path.

6. Case Studies

Our results demonstrate the prevalence of the weak-link vulnerability among popular VPN applications. However, we ask the following question: *Can a censor abuse the weak-link vulnerability to cause a large-scale disruption of VPN infrastructures, including non-vulnerable servers?* In this section, we show how the weak-link vulnerability can be abused by an adversary to identify patterns from vulnerable infrastructures and leverage them to discover and potentially block probe-resistant servers.

6.1. Methodology

We investigate how a censor can leverage weak-link-vulnerable servers to uncover a VPN provider's broader infrastructure, including servers that resist active probing. A key challenge for the censor is its limited visibility into VPN infrastructure. However, as demonstrated in our previous analysis, the coexistence of strong and weak protocols provides an alternative discovery mechanism. By actively probing fingerprintable protocols, a censor can identify an initial set of vulnerable VPN servers without requiring access to the provider's internal infrastructure or application-specific information. We examine how publicly observable metadata associated with these servers can reveal shared infrastructural patterns, allowing the censor to expand its discovery beyond directly fingerprintable endpoints. Figure 9 illustrates this process.

Step 1: Metadata Collection. The censor starts with a set of weak-link VPN servers previously discovered through active probing. These servers provide an initial foothold for identifying the broader VPN infrastructure without requiring access to application-specific APIs or server lists. The censor then collects publicly observable network metadata from these servers, including domain names, reverse DNS records, and TLS certificates from additional open services.

Step 2: Pattern Identification. Next, the censor analyzes the collected metadata to identify common infrastructural patterns. For example, servers may share naming conventions such as `node1.examplevpn.com` and `node2.examplevpn.com`, allowing the censor to derive a wildcard-based pattern such as `*.examplevpn.com`. Notably, these patterns can be identified from a limited set of vulnerable servers (e.g., two IP addresses), without requiring previous knowledge of the provider's infrastructure.

Step 3: Pattern-Based Blocking. Finally, the censor leverages the identified patterns to expand its visibility into the provider's infrastructure. By collecting and analyzing the same types of metadata (e.g., DNS records and TLS certificates) from additional servers, the censor can identify additional VPN infrastructure sharing the same patterns, including VPN servers that exclusively host probe-resistant protocols. Consequently, the censor can expand its blocklist beyond weak-link servers, potentially disrupting VPN connections that would otherwise resist active probing.

Experimental Validation. To demonstrate the feasibility of this approach, we investigate whether metadata associated with a partial set of weak-link-vulnerable servers can expose a VPN provider's broader infrastructure. We manually establish VPN connections using strong protocols (e.g., WireGuard) to obtain initial server IP addresses and scan them on Censys to verify they are affected by the weak-link. Next, we examine their publicly observable metadata, including domain names and TLS certificates from additional open services. We then identify naming patterns and construct wildcard-based queries, which are submitted to Censys to enumerate additional endpoints sharing the same infrastructural identifiers.

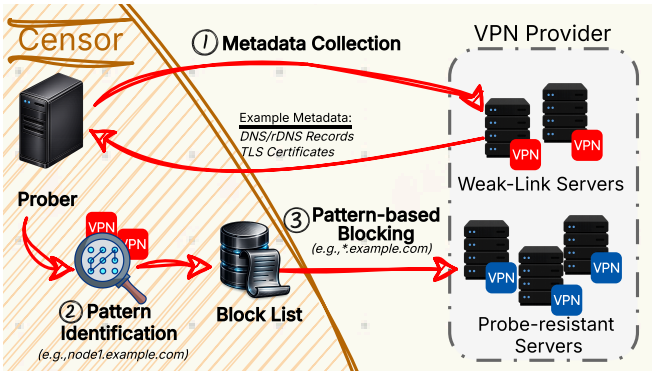


Figure 9: Pattern-based infrastructure blocking enabled by the weak-link vulnerability. A censor first probes weak-link servers and collects externally visible metadata, such as DNS/rDNS records and TLS certificates. The censor then extracts provider-specific patterns from this metadata and uses them to expand a blocklist, allowing both weak-link and otherwise probe-resistant servers belonging to the same VPN provider to be identified and potentially blocked.

We emphasize that these manual connections are part of our experimental setup and are not a capability assumed for the censor. From the adversary's perspective, the initial weak-link servers are obtained through active probing of weak protocols, as described above. Once these servers are identified, the subsequent steps rely exclusively on publicly observable network metadata, including information from additional open services. We examine whether the resulting patterns expose additional servers that are not directly fingerprintable, demonstrating how a partial set of vulnerable endpoints can reveal a provider's broader infrastructure.

6.2. Findings

To demonstrate our findings in practice, we investigate three additional popular VPN applications from the initial app corpus (which were initially filtered) that offer strong protocols. We summarize our case studies' results in Table 7.

TunnelBear. We identify the naming pattern `*.lazerpenguin.com` from reverse DNS records, which reveals a total of 16K hosts, of which 7.9K (49.38%) are identified as VPN infrastructure. Strikingly, the remaining 8.1K servers (50.63%) are not directly identified through open VPN services, yet share the same infrastructural identifiers. Consequently, these servers can also be impacted by blocking based on the patterns identified from the initial weak-link servers.

Interestingly, we find that TunnelBear shares IPs with our McAfee dataset, as both services are managed by the same parent organization [75]. However, this shared infrastructure is not disclosed by either app. Consequently, any weak-link introduced by one provider can potentially endanger not only its own users but those of the other application as well. Collectively, TunnelBear and McAfee

Application	Infrastructural Pattern	Relevant Metadata	Detected	
			Total	PR
VyprVPN	`*.vyprvpn.com`	Reverse DNS	108	28
Norton VPN	`*.gen-vpn.com`	SAN in TLS	3.9K	2.2K
TunnelBear	`*.lazerpenguin.com`	Reverse DNS	16K	8.1K

TABLE 7: Detection summary for case studies. For each app, we note the pattern found in network metadata and the size of their infrastructure. The *Total* and *PR* columns show the number of servers discovered and the probe-resistant ones among them, respectively.

have over 60 million installations, highlighting the potential real-world impact of shared infrastructure.

Norton VPN. We identify a pattern in the Subject Alternative Name (SAN) of a TLS certificate for an HTTP service on the server: `*.gen-vpn.com`. The query enumerates 3.9K servers, of which 1.7K (43.59%) are identified as VPN infrastructure. Notably, the remaining 2.2K servers (56.41%) share the same certificate-based identifier but are not directly identified through weak VPN services. Therefore, a censor leveraging this pattern can extend its blocklist to additional servers that would otherwise remain difficult to discover through protocol-specific probing.

VyprVPN. The reverse DNS resolution reveals the following pattern in the domain name: `*.vyprvpn.com`. We then query this pattern on Censys and find a total of 108 hosts, 80 (74.07%) of which are labeled as VPNs, while the remaining 28 are not directly identified through VPN fingerprinting. Consequently, a censor leveraging this pattern can expand its blocklist beyond the 80 directly identifiable servers, potentially affecting the additional 28 hosts that share the same infrastructural identifiers.

Takeaway: We show how the weak-link vulnerability can be abused to expand infrastructure discovery beyond directly fingerprintable VPN servers. By identifying common patterns in publicly observable metadata, censors can potentially discover and block additional servers, including those hosting probe-resistant protocols.

7. Discussion

Recommendations. We recommend that VPN providers remove weak VPN protocols from their infrastructure. While we acknowledge that some providers continue to support weak protocols for compatibility and flexibility, we argue that strong protocols can now satisfy the vast majority of these use cases without exposing users to unnecessary risk.

In cases where complete removal is not feasible due to operational constraints, such as backward compatibility requirements, we recommend isolating strong protocols with dedicated infrastructure. This approach allows providers to continue supporting outdated client versions while preventing weak protocols from undermining the censorship resistance of infrastructure serving up-to-date users.

Additionally, providers deploying double-hop for restrictive-network environments should ensure that entry servers resist fingerprinting, since these are the endpoints directly contacted by users. Exit-node protection remains relevant when adversaries can observe or interfere with later segments of the communication path, but its blocking implications depend on the adversary's network visibility.

Real Impact. Our impact goes beyond the 13 investigated VPN applications, reflecting an ecosystem-level risk. The application of our findings, exhibited through case studies, reinforces the ability of the weak-link vulnerability to affect servers that are thought to be secure. The patterns within their infrastructural attributes and external artifacts (e.g., DNS patterns or TLS certificates) further show that the weak-link problem is not confined to the studied apps.

Server Overfetching & API Security. Our methodology highlights that VPN backend APIs should be treated as *critical infrastructure*, particularly when applications overfetch comprehensive server inventories. While prior work has shown that adversaries can collect VPN server IPs through repeated connect/disconnect cycles [76], our results demonstrate that API reversing and request simulation can expose complete server lists at substantially greater scale. In our dataset, multiple applications exposed server-list endpoints without authentication, while others relied on weak authentication practices, such as long-lived bearer tokens or incomplete revocation after subscription expiration.

We recommend that providers follow least-privilege principles in server discovery by exposing only location metadata required for UI population and retrieving protocol-specific details (e.g., IPs, ports, and configurations) only when a connection is initiated. Additionally, providers should enforce strong authentication and authorization, bind tokens to user state or device sessions, implement short token lifetimes and reliable revocation, and deploy rate-limiting and abuse monitoring. Although an additional encryption layer over TLS can increase the cost of large-scale replication and enumeration, it should complement—not replace—secure API access controls.

Limitations. This study faced several limitations: First, our app corpus is restricted by accessibility. Some VPNs require payment before exposing the infrastructure data needed for analysis, and some implement strong anti-analysis measures. Second, some VPN providers respond with encrypted response payloads over already TLS-encrypted sessions. A longitudinal collection in those cases would require app-specific cryptographic re-implementation, which is beyond our scope. Third, the methodology is difficult to scale across a much larger number of applications because accurate API reversing remains labor-intensive. Finally, our measurements represent a lower bound of the weak-link vulnerability's prevalence, as Censys may fail to identify fingerprintable VPN protocols running on certain ports. Additionally, VPN providers not included in our study may exhibit similar co-deployment practices, potentially extending the prevalence of this vulnerability across the VPN ecosystem.

Responsible Disclosure. To improve the security pos-

ture of the commercial VPN ecosystem, we responsibly disclosed our findings to all 13 affected VPN providers. At this time, two providers acknowledged our recommendations. We will follow up with them to provide validation of fixes.

8. Related Work

VPN, Proxy, and Tor Fingerprinting. Censors and circumventors have been engaged in an ongoing arms race. Initially, many studies focused on the conflict between the Great Firewall and Tor [16], [17], [77]. Later, censors began blocking other circumvention tools, such as obfuscated proxy protocols, which were designed to provide lightweight solutions against these restrictions [15], [19], [78]. Frolov et al. [19] examined the feasibility of censors detecting probe-resistant proxies, such as obfs4, Lampshade, and Shadowsocks. They identified distinctive TCP behaviors in probe-resistant proxies, including connection termination and timeout patterns, that enable fingerprinting through a small number of active probes. Alice et al. [15] unveiled how the Great Firewall detects and blocks Shadowsocks through active probing and traffic analysis techniques while also suggesting defenses against them. Subsequently, developers introduced Shadowsocks 2022, which mitigates these detection methods [79].

With the rise of VPNs as circumvention tools, various studies have focused on identifying VPN servers and connections [28]. Xue et al. [18] analyzed how OpenVPN remains vulnerable to fingerprinting by censors in real-world deployments. They concluded that OpenVPN, under most configurations, remained feasible to fingerprint even for small ISPs. Rajore et al. [76] extended this line of work to WireGuard, IKEv2, and proprietary VPN protocols across seven commercial VPN providers, identifying patterns for detection and obfuscation implemented by other protocols.

Prior work has also focused on identifying VPN traffic through passive analysis [25], [80]. For example, Wang et al. [81] proposed VPNCHECKER, which detects VPN servers through graph-represented behaviors and deployed it in an ISP setting. While most prior work has focused on finding protocol-specific patterns for fingerprinting, we demonstrate how censors can effectively block servers without requiring sophisticated probes. We show that identifying at least one weak protocol is sufficient for a censor to block an entire server and affect strong protocols.

VPN Security. Since VPNs make users transfer trust from their ISP to their VPN provider, many works have examined the security implications of this trust model [56], [67], [82], [83]. Ramesh et al. [82] presented VPNALYZER and performed a large-scale audit, finding that desktop VPNs exhibit various misconfigurations that could lead to traffic leaks. Wang et al. [67] extended this direction to mobile applications with MVPNALYZER and identified similar behavior in Android apps, highlighting the lack of obfuscation in VPN connections. These studies emphasize the need for strong configurations to guarantee security promises.

Other studies have focused on identifying infrastructure-wide vulnerabilities [84]–[87]. For example, Mixon-Baca et al. [84] introduced the *port shadow* attack, in which attackers could exploit internal routing rules to hijack user traffic. However, these works overlook infrastructure-level inconsistencies. Our work shows that providers introduce an infrastructure-wide vulnerability by deploying strong and weak protocols on the same servers.

9. Conclusion

In this work, we show that the weak-link vulnerability is an ecosystem-wide issue: thousands of VPN servers co-deploy weak and secure protocols within the same infrastructure. We design a data collection pipeline to measure the prevalence of this vulnerability and identify deployment practices that expose servers hosting probe-resistant protocols. We also assess the practicality of leveraging our findings to impact otherwise probe-resistant servers. Users worldwide rely on VPNs to circumvent censorship. Yet, the ease with which many "obfuscated" deployments remain detectable by censors raises serious concerns. We recommend that VPN providers discontinue weak protocols or isolate them from servers hosting stronger options.

Acknowledgments

The authors thank the anonymous reviewers for their constructive feedback. This research was supported by the National Science Foundation under Grant Numbers CNS-2452884 and CNS-2452883.

References

[1] H. Le, A. Huremagic, K. Wang, R. Ensafi, and R. S. Raman, "CenRL: A Framework for Performing Intelligent Censorship Measurements," in *2026 IEEE Symposium on Security and Privacy (S&P)*, 2026.

[2] The Guardian, "Alarming rise in regional internet censorship in china, study finds," https://www.theguardian.com/world/2025/may/24/alarming-rise-in-regional-internet-censorship-in-china-study-finds, 2025, accessed: 2026-06-10.

[3] Al Jazeera, "India expands censorship powers, lets lower officials demand takedowns," https://www.aljazeera.com/features/2025/9/8/india-expands-censorship-powers-lets-lower-officials-demand-takedowns, 2025, accessed: 2026-06-10.

[4] VPNMentor, "Vpn surge in brazil," https://www.vpnmentor.com/news/vpn-surge-brazil/, 2025, accessed: 2026-06-10.

[5] Freedom House, "Freedom on the net 2022: United arab emirates," https://freedomhouse.org/country/united-arab-emirates/freedom-net/2022, 2022, accessed: 2026-06-10.

[6] Amnesty International, "Pakistan: Mass surveillance and censorship machine is fueled by chinese, european, emirati and north american companies," https://www.amnesty.org/en/latest/news/2025/09/pakistan-mass-surveillance-and-censorship-machine-is-fueled-by-chinese-european-emirati-and-north-american-companies/, 2025, accessed: 2026-06-10.

[7] A. Ortwein, A. Ablove, A. Huremagic, L. Chang, V. Fortuna, and R. Ensafi, "Amplifying user voices to rally censorship investigation," in *2026 IEEE Symposium on Security and Privacy (S&P)*, 2026.

[8] Digital Watch Observatory, "Turkey bans access to 16 vpn providers in effort to censor blocked news websites," https://dig.watch/updates/turkey-bans-access-to-16-vpn-providers-in-effort-to-censor-blocked-news-websites, 2023, accessed: 2026-04-21.

[9] C. Castro, "India weighs stricter VPN regulations to stop users from bypassing internet blocks," https://www.techradar.com/vpn/vpn-privacy-security/india-weighs-stricter-vpn-regulations-to-stop-users-from-bypassing-internet-blocks, 2026, accessed: 2026-10-12.

[10] TechRadar, "Russia's solution to its vpn crackdown breaking the internet? a state-owned vpn," https://www.techradar.com/vpn/vpn-privacy-security/russias-solution-to-its-vpn-crackdown-breaking-the-internet-a-state-owned-vpn, 2026, accessed: 2026-06-10.

[11] European Country of Origin Information Network (ecoi.net), "Cuba's internet: Blocked pages and chinese tech," https://www.ecoi.net/de/dokument/2042706.html, 2021, document No. 2042706, Accessed: 2026-06-10.

[12] TechRadar, "After tiktok, venezuela blocks vpn websites," https://www.techradar.com/vpn/vpn-privacy-security/after-tiktok-venezuela-blocks-vpn-webistes, 2025, accessed: 2026-06-10.

[13] G. Phillips, "VPN usage soars in Uganda as internet access returns – but privacy tools may be under attack," https://www.tomsguide.com/computing/vpns/vpn-usage-soars-in-uganda-as-internet-access-returns-but-privacy-tools-may-be-under-attack, Jan. 2026, accessed: 2026-10-12.

[14] M. Wu, J. Sippe, D. Sivakumar, J. Burg, P. Anderson, X. Wang, K. Bock, A. Houmansadr, D. Levin, and E. Wustrow, "How the great firewall of china detects and blocks fully encrypted traffic," in *32nd USENIX Security Symposium (USENIX Security 23)*, 2023.

[15] Alice, Bob, Carol, J. Beznazwy, and A. Houmansadr, "How China Detects and Blocks Shadowsocks," in *Proceedings of the ACM Internet Measurement Conference (IMC)*, 2020.

[16] R. Ensafi, D. Fifield, P. Winter, N. Feamster, N. Weaver, and V. Paxson, "Examining How the Great Firewall Discovers Hidden Circumvention Servers," in *Proceedings of the 2015 Internet Measurement Conference (IMC)*, 2015.

[17] R. Ensafi, P. Winter, A. Mueen, and J. R. Crandall, "Analyzing the Great Firewall of China Over Space and Time," in *Proceedings on Privacy Enhancing Technologies (PETS)*, 2015.

[18] D. Xue, R. Ramesh, A. Jain, M. Kallitsis, J. A. Halderman, J. R. Crandall, and R. Ensafi, "OpenVPN is Open to VPN Fingerprinting," in *The 31st USENIX Security Symposium (USENIX Security 22)*, 2022.

[19] S. Frolov, J. Wampler, and E. Wustrow, "Detecting Probe-resistant Proxies," in *The Network and Distributed System Security Symposium (NDSS)*, 2020.

[20] "Hardening OpenVPN Security — openvpn.net," https://openvpn.net/community-docs/hardening-openvpn-security.html, [Accessed 05-02-2026].

[21] J. A. Donenfeld, "Wireguard: Next generation kernel network tunnel." in *The Network and Distributed System Security Symposium (NDSS)*, 2017.

[22] M. T. Khan, J. Deblasio, G. M. Voelker, A. C. Snoeren, C. Kanich, and N. Vallina-Rodriguez, "An Empirical Analysis of the Commercial VPN Ecosystem," 2018. [Online]. Available: https://doi.org/10.1145/3278532.

[23] Proton VPN, "Bypass censorship with a vpn," https://protonvpn.com/vpn/censorship, accessed: 2026-06-04.

[24] TunnelBear, "Tunnelbear: Secure vpn service," https://www.tunnelbear.com/, accessed: 2026-06-04.

[25] D. Xue, M. Kallitsis, A. Houmansadr, and R. Ensafi, "Fingerprinting Obfuscated Proxy Traffic with Encapsulated TLS Handshakes," in *33rd USENIX Security Symposium (USENIX Security 24)*, 2024.

[26] D. Xue, R. Ramesh, V. S. S, L. Evdokimov, A. Viktorov, A. Jain, E. Wustrow, S. Basso, and R. Ensafi, "Throttling twitter: an emerging censorship technique in russia," in *Proceedings of the 21st ACM Internet Measurement Conference (IMC)*, 2021.

[27] S. Khattak, D. Fifield, S. Afroz, M. Javed, S. Sundaresan, V. Paxson, S. J. Murdoch, and D. McCoy, "Do you see what I see? differential treatment of anonymous users," in *The Network and Distributed System Security Symposium (NDSS)*, 2016.

[28] A. Maghsoudlou, L. Vermeulen, I. Poese, and O. Gasser, "Characterizing the VPN Ecosystem in the Wild," in *International Conference on Passive and Active Network Measurement (PAM)*, 2023.

[29] ExpressVPN, "PPTP and L2TP manual configurations no longer supported," https://www.expressvpn.com/support/troubleshooting/manual-pptp-not-supported/, Dec. 2025, accessed: 2026-02-05.

[30] CyberGhost VPN, "Network updates: we are no longer supporting PPTP and L2TP," https://www.cyberghostvpn.com/privacyhub/l2tp_pptp_discontinued/, Jan. 2025, accessed: 2026-02-05.

[31] D. Markuson, "NordVPN will discontinue two VPN protocols on Dec. 1st," https://nordvpn.com/blog/l2tp-pptp-protocol-update/, Oct. 2018, accessed: 2026-02-05.

[32] OpenVPN Community Documentation, "OpenVPN Protocol," https://openvpn.net/community-docs/openvpn-protocol.html, 2025, accessed: 2026-02-05.

[33] XTLS Project, "All-in-One Fallbacks with Nginx," https://github.com/XTLS/Xray-examples/blob/main/All-in-One-fallbacks-Nginx/README.md, 2024, accessed: 2026-04-26.

[34] Shadowsocks-NET, "Shadowsocks 2022 Edition: Detection Prevention (Section 3.1.3)," https://github.com/Shadowsocks-NET/shadowsocks-specs/blob/main/2022-1-shadowsocks-2022-edition.md#313-detection-prevention, 2022, accessed: 2026-05-05.

[35] NordVPN, "NordWhisper Protocol: What It Is and How It Works," https://nordvpn.com/blog/nordwhisper-protocol/, 2024, accessed: 2026-04-26.

[36] Kaufman, Charlie and Hoffman, Paul and Nir, Yoav and Eronen, Pasi and Kivinen, Tero, "Internet Key Exchange Protocol Version 2 (IKEv2), Section 1," https://datatracker.ietf.org/doc/html/rfc7296#section-1, 2014.

[37] P. Wouters, "Deprecation of the Internet Key Exchange Version 1 (IKEv1) Protocol and Obsoleted Algorithms," https://datatracker.ietf.org/doc/rfc9395/, 2023, iETF Proposed Standard, accessed: 2026-02-05.

[38] D. Harkins and D. Carrel, "The internet key exchange (ike)," https://datatracker.ietf.org/doc/html/rfc2409, Nov. 1998, rFC 2409, accessed June 2026.

[39] F. Ali, "PPTP and L2TP deprecation: A new era of secure connectivity," https://techcommunity.microsoft.com/blog/windowsservernewsandbestpractices/pptp-and-l2tp-deprecation-a-new-era-of-secure-connectivity/4263956, Oct. 2024, accessed: 2026-02-05.

[40] W. Townsley, A. Valencia, G. Rubens, G. Pall, G. Zorn, and B. Palter, "Layer two tunneling protocol "l2tp"," https://datatracker.ietf.org/doc/html/rfc2661#section-3.0, Aug. 1999, rFC 2661, Section 3.0: Protocol Overview, accessed June 2026.

[41] K. Hamzeh, G. Pall, W. Verthein, J. Taarud, W. Little, and G. Zorn, "Point-to-point tunneling protocol (pptp)," https://datatracker.ietf.org/doc/html/rfc2637, Jul. 1999, rFC 2637, accessed June 2026.

[42] Security.org, "2026 Research Into VPN Usage (Updated Statistics)," https://www.security.org/vpn/statistics/, 2026, accessed: 2026-06-01.

[43] StatCounter Global Stats, "Mobile Operating System Market Share Worldwide," https://gs.statcounter.com/os-market-share/mobile/worldwide, 2026, accessed: 2026-06-01.

[44] Genymotion, "Genymotion – android emulator for development and testing," https://www.genymotion.com/, 2026, accessed: 2026-02-05.

[45] Y. Hong, Y. Hu, C.-M. Lai, S. Felix Wu, I. Neamtiu, P. McDaniel, P. Yu, H. Cam, and G.-J. Ahn, "Defining and detecting environment discrimination in android apps," in *International Conference on Security and Privacy in Communication Systems*, 2017.

[46] A. Cortesi, M. Hils, T. Kriechbaumer, and contributors, "mitmproxy: A free and open source interactive HTTPS proxy," https://mitmproxy.org/.

[47] "Frida: A world-class dynamic instrumentation toolkit," https://frida.re/, [Accessed 04-02-2026].

[48] F. Hantke, S. Roth, R. Mrowczynski, C. Utz, and B. Stock, "Where Are the Red Lines? Towards Ethical Server-Side Scans in Security and Privacy Research," in *2024 IEEE Symposium on Security and Privacy (S&P)*, 2024.

[49] D. Dittrich and E. Kenneally, "The Menlo Report: Ethical Principles Guiding Information and Communication Technology Research," https://www.dhs.gov/sites/default/files/publications/CSD-MenloPrinciplesCORE-20120803_1.pdf, U.S. Department of Homeland Security, 2012, accessed: 2026-06-01.

[50] M. Antonakakis, T. April, M. Bailey, M. Bernhard, E. Bursztein, J. Cochran, Z. Durumeric, J. A. Halderman, L. Invernizzi, M. Kallitsis, D. Kumar, C. Lever, Z. Ma, J. Mason, D. Menscher, C. Seaman, N. Sullivan, K. Thomas, and Y. Zhou, "Understanding the Mirai Botnet," in *26th USENIX Security Symposium (USENIX Security 17)*, 2017.

[51] A. P. Felt, R. Barnes, A. King, C. Palmer, C. Bentzel, and P. Tabriz, "Measuring HTTPS Adoption on the Web," in *26th USENIX Security Symposium (USENIX Security 17)*, 2017.

[52] X. Feng, Q. Li, H. Wang, and L. Sun, "Acquisitional rule-based engine for discovering Internet-of-Things devices," in *27th USENIX Security Symposium (USENIX Security 18)*, 2018.

[53] R. Sundara Raman, P. Shenoy, K. Kohls, and R. Ensafi, "Censored Planet: An Internet-wide, Longitudinal Censorship Observatory," in *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2020.

[54] P. Pearce, R. Ensafi, N. Feamster, N. Weaver, and V. Paxson, "Global Measurement of DNS Manipulation," in *26th USENIX Security Symposium (USENIX Security 17)*, 2017.

[55] Z. Durumeric, D. Adrian, A. Mirian, M. Bailey, and J. A. Halderman, "A Search Engine Backed by Internet-Wide Scanning," in *22nd ACM Conference on Computer and Communications Security (CCS)*, 2015.

[56] R. Ramesh, A. Vyas, and R. Ensafi, ""All of them claim to be the best": Multi-perspective study of VPN users and VPN providers," in *32nd USENIX Security Symposium (USENIX Security 23)*, 2023.

[57] A. Hawkins, "Chinese programmer ordered to pay 1m yuan for using virtual private network," https://www.theguardian.com/world/2023/oct/09/chinese-programmer-ordered-to-pay-1m-yuan-for-using-virtual-private-network, Oct. 2023, accessed: 2026-05-07.

[58] Proton VPN Support, "How to download wireguard configuration files," https://protonvpn.com/support/wireguard-configurations, 2026, accessed: 2026-02-05.

[59] ZipVPN, "V2ray vs xray protocol," https://www.zipvpn.net/blog/article/v2ray-xray-protocol.html, accessed: 2026-06-09.

[60] PureVPN Support, "Discontinuation of PPTP, L2TP, and SSTP Protocols," https://support.purevpn.com/en_US/purevpn/protocol-discontinuation, 2021, accessed: 2026-02-05.

[61] Surfshark, "Surfshark is removing the ikev2 protocol from windows," https://surfshark.com/blog/surfshark-removes-ikev2-windows, Jul. 2022, accessed: 2026-02-05.

[62] Proton VPN, "Discontinuing support for ikev2 and openvpn on ios and macos," https://protonvpn.com/support/discontinuing-ikev2-openvpn-macos-ios, 2024, accessed: 2026-02-05.

[63] NordVPN, "What is a multi-hop vpn, and should you have one?" https://nordvpn.com/blog/multi-hop-vpn/, 2025, accessed: 2026-05-15.

[64] O. Akgul, R. Roberts, M. Namara, D. Levin, and M. L. Mazurek, "Investigating Influencer VPN Ads on YouTube," in *2022 IEEE Symposium on Security and Privacy (S&P)*, 2022.

[65] O. Akgul, R. Roberts, E. Shroyer, D. Levin, and M. L. Mazurek, "As Advertised? Understanding the Impact of Influencer VPN Ads," in *34th USENIX Security Symposium (USENIX Security 25)*, 2025.

[66] B. Mixon-Baca, J. Knockel, and J. Crandall, "Hidden Links: Analyzing Secret Families of VPN Apps," in *Proceedings of the Free and Open Communications on the Internet (FOCI)*, 2025.

[67] W. Wang, A. Ortwein, E. Sobrados, R. Stanley, P. K. Sharma, A. Anwar, and R. Ensafi, "MVPNalyzer: An Investigative Framework for Auditing the Security & Privacy of Mobile VPNs," in *The Network and Distributed System Security Symposium (NDSS)*, 2026.

[68] PureVPN, "Ikev2 vpn protocol: What it is and how it works," https://www.purevpn.com/what-is-vpn/protocols/ikev2, 2026, accessed: 2026-02-05.

[69] NordLayer, "Ikev2 vpn protocol: What it is and how it works," https://nordlayer.com/learn/vpn/ikev2/, 2026, accessed: 2026-02-05.

[70] Surfshark. (2025) Vpn protocols explained. [Online]. Available: https://surfshark.com/blog/vpn-protocols

[71] Security.org, "Types of vpn protocols: Explanation and comparison," https://www.security.org/vpn/protocols/, Nov. 2025, accessed: 2026-06-01.

[72] Shadowsocks Contributors, "Shadowsocks: A fast tunnel proxy that helps you bypass firewalls," https://shadowsocks.org/, 2026, accessed: 2026-06-01.

[73] Project X, "Xray Server Guide," https://xtls.github.io/en/document/level-0/ch07-xray-server.html, 2026, accessed: 2026-06-09.

[74] C. Castro, "Vpns have adapted: How blancvpn and vpn liberty are dodging russia's vpn blocking technology to allow russians access to telegram without losing everyday services," https://www.techradar.com/vpn/vpn-privacy-security/vpns-have-adapted-how-blancvpn-and-vpn-liberty-are-dodging-russias-vpn-blocking-technology-to-allow-russians-access-to-telegram-without-losing-everyday-services, 2026, accessed 2026-06-10.

[75] TunnelBear, "Tunnelbear joins mcafee," https://www.tunnelbear.com/blog/tunnelbear_joins_mcafee/, Mar. 2018, accessed: 2026-06-10.

[76] T. Rajore, J. S, A. Gupta, K. Gambhir, A. Prithvi, and S. Chakravarty, "VPN or Vpwn? How Afraid Should You be of VPN Traffic Identification?" in *2025 9th Network Traffic Measurement and Analysis Conference (TMA)*, 2025.

[77] S. Matic, C. Troncoso, and J. Caballero, "Dissecting Tor Bridges: A Security Evaluation of Their Private and Public Infrastructures," in *The Network and Distributed System Security Symposium (NDSS)*, 2017.

[78] D. Xue, A. Ablove, R. Ramesh, G. K. Danciu, and R. Ensafi, "Bridging barriers: A survey of challenges and priorities in the censorship circumvention landscape," in *33rd USENIX Security Symposium (USENIX Security 24)*, 2024.

[79] Shadowsocks-NET, "Shadowsocks 2022 Edition: Secure L4 tunnel with symmetric encryption," https://github.com/Shadowsocks-NET/shadowsocks-specs/blob/main/2022-1-shadowsocks-2022-edition.md, 2022, protocol specification. Accessed: 2026-10-12.

[80] D. Xue, R. Stanley, P. Kumar, and R. Ensafi, "The Discriminative Power of Cross-layer RTTs in Fingerprinting Proxy Traffic," in *The Network and Distributed System Security Symposium (NDSS)*, 2025.

[81] C. Wang, J. Yin, Z. Li, H. Xu, Z. Zhang, and Q. Liu, "Identifying VPN Servers through Graph-Represented Behaviors," in *Proceedings of the ACM Web Conference 2024*, 2024.

[82] R. Ramesh, L. Evdokimov, D. Xue, and R. Ensafi, "VPNalyzer: systematic investigation of the VPN ecosystem," in *The Network and Distributed System Security (NDSS)*, 2022.

[83] T. Bui, S. Rao, M. Antikainen, and T. Aura, "Client-side Vulnerabilities in Commercial VPNs," in *Nordic Conference on Secure IT Systems (NordSec)*, 2019.

[84] B. Mixon-Baca, J. Knockel, D. Xue, T. Ayyagari, D. Kapur, R. Ensafi, and J. R. Crandall, "Attacking connection tracking frameworks as used by virtual private networks," in *Proceedings on Privacy Enhancing Technologies (PETS)*, 2024.

[85] N. Xue, Y. Malla, Z. Xia, C. Pöpper, and M. Vanhoef, "Bypassing Tunnels: Leaking VPN Client Traffic by Abusing Routing Tables," in *32nd USENIX Security Symposium (USENIX Security 23)*, 2023.

[86] M. Ikram, N. Vallina-Rodriguez, S. Seneviratne, M. A. Kaafar, and V. Paxson, "An Analysis of the Privacy and Security Risks of Android VPN Permission-enabled Apps," in *Proceedings of the 2016 Internet Measurement Conference (IMC)*, 2016.

[87] Y. Yang, A. Wang, X. Feng, Q. Li, and K. Xu, "Invisible Adversaries: A Systematic Study of Session Manipulation Attacks on VPNs," *arXiv preprint arXiv:2604.04099*, 2026.

Appendix A. Open Science

The scripts used to collect the ground-truth VPN server information are kept private and made available to researchers upon request. Publicly releasing these scripts could enable the identification and fingerprinting of probe-resistant VPN servers, potentially putting at risk the very users this work aims to protect.

Appendix B. Research Ethics Considerations

This work identifies deployment weaknesses in commercial VPN infrastructure that could enable adversaries to fingerprint and block VPN servers. Because these findings involve vulnerabilities in deployed systems, we treated the work as security research with potential dual-use implications. We responsibly disclosed our findings and recommendations to the affected VPN providers so they could evaluate and remediate the issue. In accordance with the conference's vulnerability-disclosure expectations, we do not include provider-specific exploit automation, complete enumeration scripts, authentication material, or other artifacts that would directly enable third parties to reproduce large-scale infrastructure discovery while providers are still addressing the issue.

Our study did not involve human subjects research. We did not recruit users, interact with VPN users, monitor user traffic, collect user communications, or access any user accounts, credentials, browsing activity, subscriber records, or other personally identifiable information. All measurements were performed using researcher-controlled devices, researcher-controlled VPN client installations, publicly observable service metadata, and server information exposed to the official VPN clients. Our analysis focused on infrastructure-level deployment properties rather than user behavior or user data.

We also took steps to minimize harm to live systems. Our experiments were limited to read-only client-side requests that replicated the normal behavior of VPN applications when retrieving server configuration data. We did not attempt to compromise backend systems, modify server state, access confidential backend resources, exploit VPN servers, disrupt VPN tunnels, or interfere with normal VPN service operation. To avoid unnecessary load on provider infrastructure, our automated collection scripts introduced 5-second delays between requests in accordance with prior research. The purpose of these requests was to observe server metadata exposed to VPN clients, not to stress or attack provider backend services.

We weighed the risks of disclosure against the benefits of informing providers and the research community about a systemic weakness that affects censorship-circumvention guarantees. We believe the public benefit of documenting the vulnerability and recommending mitigations outweighs the residual risk, particularly given our responsible disclosure process and our decision to withhold operationally sensitive scripts.

Appendix C.
Meta-Review

The following meta-review was prepared by the program committee for the 2027 IEEE Symposium on Security and Privacy (S&P) as part of the review process as detailed in the call for papers.

C.1. Summary

This paper examines the issue of VPN servers hosting both protocols that resist detection by probing and those that are vulnerable to it. If a VPN server hosts both probe-resistant and probe-vulnerable protocols, then it can be discovered by its vulnerable protocols (making it susceptible to blocking by a censor). Through a series of measurements, the paper shows that protocol coexistence is common across VPN servers scanned by Censys.

C.2. Scientific Contributions

- Identifies an Impactful Vulnerability
- Provides a Valuable Step Forward in an Established Field
- Establishes a New Research Direction

C.3. Reasons for Acceptance

1) Identifies an Impactful Vulnerability, Provides a Valuable Step Forward in an Established Field, Establishes a New Research Direction: The central finding of this paper is a vulnerability leading to the discoverability of otherwise-hidden VPN servers. Of the VPN service providers examined in this paper, all but one had at least 84% of their servers vulnerable to discovery. Prior work has shown that anti-censorship bridge relays in the Tor anonymity network can be detected by the same vulnerability, but this work is the first to show impact on VPNs.